

外部排序

很高兴，我们走到了这门课，也是浙江大学数据结构与算法课程系列的最后一个章节。作为两个学期数据结构与算法的收尾，我们将集中讨论外存中的排序算法（之前的 B+ 树和倒排索引已经涉及外存中的数据结构）。外部排序的现实应用很多，例如，要统计大量文本中每个词出现的次数，可以先按词排序，让相同的词排在一起，再顺序扫描计数。即使这些数据无法一次放进内存，也可以借助外部排序完成这一步，去重和建立倒排索引时也会用到类似的方法。当然本章的介绍是非常初步的，更进一步的内容可以阅读本章参考资料。

15.1 外部排序的基本问题

前面讨论排序时，我们通常假定所有数据都能放进内存，访问任意一个元素只需常数时间。例如，希尔排序可以直接比较 $A[i]$ 和 $A[i - h_k]$ ，堆排序可以直接找到某个结点的孩子，快速排序也可以随时读取首、尾和中间位置的元素。但是，如果待排序的数据量很大，无法一次读入内存，这些操作就未必方便了。在磁带上，读到远处的数据之前必须先移动磁带；在机械硬盘上，也可能需要寻道并等待盘片转到相应位置。此时，花在读取数据上的时间可能远远超过比较本身。

因此，外部排序首先要考虑怎样减少外存读写，并尽量连续地访问数据。归并排序恰好适合这样做，因为归并时只需顺序扫描数据，再把结果依次写出。外存通常成批读写数据，我们把连续存放、作为一个读写单位的一组记录称为一块（block）。每次读入一整块，在内存中完成比较，用完后再读下一块，就能避免为每次比较单独访问外存。下面从这个办法出发，逐步改进外部排序。

15.2 二路归并

15.2.1 算法过程

我们首先介绍最基本的方法，称为二路归并（two-way merge）。假设有 N 条记录要排序，内存一次只能处理 m 条记录。我们先把数据分批处理，每次读入 m 条记录，在内存中排好序后写回外存，再读入下一批，最后不足 m 条的也作为一批处理。这样，原始数据就变成了若干个有序片段，每个片段称为一个顺串（run）。接下来只要把这些顺串归并起来，就能完成全部数据的排序。

下面用图 15.1 的例子说明这些顺串怎样存放、怎样归并。假设内存每次能排序 3 条记录，原始数据依次为 81, 94, 11, 96, 12, 35, 17, 99, 28, 58, 41, 75, 15，放在磁带 T_{a_1} 上。先读入 81, 94, 11，排序后得到第一段 11, 81, 94；再读入 96, 12, 35，得到第二段 12, 35, 96。依次处理下去，共得到五个顺串，最后一段只有 15。

归并时需要同时读取两个顺串，所以我们把这五段交替写到两条磁带 T_{b_1} 、 T_{b_2} 上，图中用 $R_1, \dots, R_5$ 按生成顺序标记它们。另外准备两条磁带 T_{a_1} 、 T_{a_2} 存放归并结果。注意虽然原始数据放

在 T_{a_1} 上，但五个初始顺串全部生成后，原始数据就不再需要了， T_{a_1} 也就可以重新用于输出。在接下来的讨论中，当所有输入处理完一次后，称为完成一趟 (pass) 归并，其中生成初始顺串算第一趟。后续的归并过程如下：

- 第二趟：把五段归并成三段。**先读取两条 b 磁带上的第一段，即 R_1 和 R_2 。比较它们当前尚未取出的最小记录，每次取较小的写到 T_{a_1} ，就依次得到 11, 12, 35, 81, 94, 96。这两段读完后，接着归并 R_3 和 R_4 ，将结果 17, 28, 41, 58, 75, 99 写到 T_{a_2} 。最后只剩 R_5 ，把它直接复制到 T_{a_1} ，作为另一段存放。这样，两条 a 磁带上分别有两段和一段，仍可继续两两归并。
- 第三趟：把三段归并成两段。**现在要读取的是两条 a 磁带。将它们各自的第一段归并，得到图中长度为 12 的顺串，写到 T_{b_1} 。 T_{a_1} 上还剩只含 15 的一段，把它复制到 T_{b_2} 。
- 第四趟：把最后两段归并成一段。**读取 T_{b_1} 上的 12 条记录和 T_{b_2} 上的 15，归并后写到 T_{a_1} ，全部数据就排好序了。

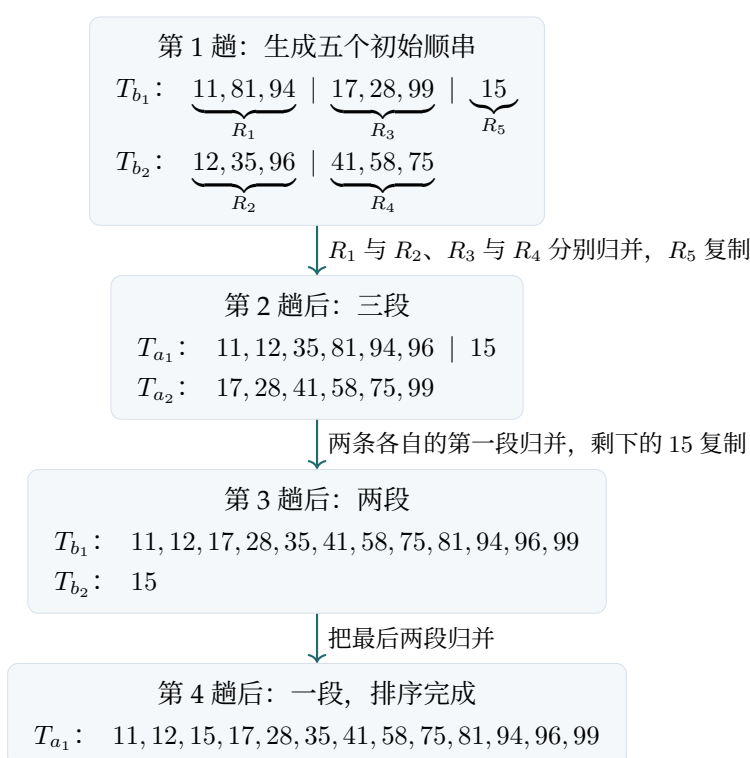


图 15.1 每次在内存中排序 3 条记录，再用四条磁带完成二路归并，竖线分隔同一条上的不同顺串。

一般的情况下处理思路是一致的：每次从两条输入磁带各取下一段，归并结果轮流写到两条输出磁带，落单的一段直接复制。此后将刚写好的两条磁带从头读取，把结果写回另一组磁带，重复这一过程直到只剩一个顺串为止。实现时，我们还需要维护一个顺串目录，保存每个顺串的长度和在磁带上的位置，以便归并时知道每段的起止位置。

15.2.2 趟数

下面计算一般二路归并需要的趟数。设待排序的数据共有 $N > 0$ 条记录，每次处理至多 m 条，最初便有 $r_0 = \lceil N/m \rceil$ 个顺串。记完成 t 趟归并后剩下 r_t 段（这里的 t 不包含生成初始顺串的一趟），由于每趟将两段合成一段，落单的一段直接复制，有 $r_{t+1} = \lceil \frac{r_t}{2} \rceil$ ，因此 $r_t = \lceil \frac{r_0}{2^t} \rceil$ 。要让最后只剩一

段, 就需要 $2^t \geq r_0$, 所以需要 $\lceil \log_2 r_0 \rceil$ 趟归并。加上生成初始顺串的一趟, 总趟数为

$$p = 1 + \lceil \log_2 r_0 \rceil.$$

例题 15.1. 假设需要对 1000 万条记录排序, 每条占 128 字节, 可用于内部排序的内存为 4 MB。用上面的二路归并方法, 需要多少趟才能排好序? 这里按十进制计算, $1 \text{ MB} = 10^6$ 字节。

解答. 内存每次可以排序 $4000000/128 = 31250$ 条记录, 因此初始有 $10^7/31250 = 320$ 个顺串。归并需要 $\lceil \log_2 320 \rceil = 9$ 趟, 加上生成顺串的一趟, 共 10 趟。□

15.3 多路归并

接下来的讨论都是针对最基本的二路归并的优化。先考虑怎样通过增加归并路数来减少趟数, 从而减少外存读写。二路归并每次只能把两段合成一段, 如果一次合并 k 段, 每趟就能把段数从 r 减到 $\lceil r/k \rceil$, 连同生成初始顺串, 总趟数就变为 $1 + \lceil \log_k r_0 \rceil$ 。例如, 将图 15.1 中的五个初始顺串按生成顺序分配到三条输入磁带上, 第一趟归并就可以合并前三段, 再合并剩下的两段, 得到长度分别为 9 和 4 的两个顺串。接下来再归并一趟即可完成排序, 连同生成顺串共需 3 趟, 比二路归并少一趟。

一个实现的细节是, 由于我们每次要选出最小的记录写入输出磁带, 因此可以在内存中维护一个大小为 k 的最小堆。每次从堆顶取出最小记录, 写入输出磁带后, 再把该记录所在顺串的下一条记录插入堆中, 从而保证输出顺序正确, 并且能在 $O(\log k)$ 时间内完成每次选择。

在多路归并中, 我们需要沿用前面的磁带分组方法, 准备 k 条输入磁带和 k 条输出磁带, 把每次归并的结果循环分配到输出磁带上, 一趟结束后交换两组磁带的用途, 这称为平衡多路归并。因此, 这个方法共需要使用 $2k$ 条磁带。

15.4 多相归并

15.4.1 二路多相归并

增加归并路数会用到更多磁带, 前面的平衡 k 路归并就需要 $2k$ 条磁带, 如果 k 较大, 所需的磁带数量可能太多。基于此我们讨论另一种优化方案: 多相合并 (polyphase merge), 其中从开始归并到其中一条磁带上的输入用完的这一段过程, 称为一相。在接下来的讨论中我们可以看到, 这一方法能够大大减小归并需要的磁带数目: k 路合并只需要 $k+1$ 条磁带。

以三条磁带上的二路归并为例, 考虑有 34 个顺串且只有 3 条磁带的情况。若把 34 个顺串平均分成 17 段和 17 段, 全部合并到第三条后, 就只剩这条磁带上 17 段。要继续归并, 必须先把其中一些段复制到另一条, 例如分成 8 段和 9 段。这样的复制没有减少顺串数量, 却增加了读写工作。

如果最初分成 21 段和 13 段, 就可以避免这次重新分配。第一相合并 13 组后, 较短的输入磁带用完, 另一条还剩 8 段, 恰好可以与刚生成的 13 段一起作下一相的输入。照此继续, 就得到表 15.1 的过程。例如, 第二相开始时, T_1 有 13 段, T_2 有 8 段。把两边各 8 段合并后, T_1 剩下 5 段, T_3 得到 8 个新顺串, T_2 空出后成为下一相的输出磁带。表中的其余各相也按这个规则进行, 最后恰好只剩一个顺串。

表 15.1 按 21 + 13 分配的归并过程。

相	输入 → 输出	T_1	T_2	T_3
初始	—	0	21	13
1	$T_2, T_3 \rightarrow T_1$	13	8	0
2	$T_1, T_2 \rightarrow T_3$	5	0	8
3	$T_1, T_3 \rightarrow T_2$	0	5	3
4	$T_2, T_3 \rightarrow T_1$	3	2	0
5	$T_1, T_2 \rightarrow T_3$	1	0	2
6	$T_1, T_3 \rightarrow T_2$	0	1	1
7	$T_2, T_3 \rightarrow T_1$	1	0	0

表 15.2 按 22 + 12 分配的归并过程。

相	本相合并组数	T_1	T_2	T_3
初始	—	0	22	12
1	12	12	10	0
2	10	2	0	10
3	2	0	2	8
4	2	2	0	6
5	2	0	2	4
6	2	2	0	2
7	2	0	2	0

如上选取的初始段数为什么合理？我们可以先看一个不够合理的例子：把 34 段分成 22 段和 12 段，过程见表 15.2。这次最终有两段留在同一条上，仍然需要先把其中一段复制到另一条，才能完成最后的合并。原因可以从段数的变化看出来。若两条分别有 $a \geq b$ 段，一相之后两条非空磁带上的段数就变成 $a - b$ 和 b 。注意这样的处理使得每一步都保持最大公约数不变。由于 $\gcd(22, 12) = 2$ ，只按这个规则归并，最后就会剩下两段。

前面选用的 21 和 13 是两个相邻的斐波那契数。它们的好处在于，每次相减后又得到更小的相邻两项： $(F_j, F_{j-1}) \rightarrow (F_{j-1}, F_{j-2})$ 。不断重复，就会到达 $(1, 0)$ ，全程无需重新分配。当然根据前面的分析，只要开始两条磁带分配的顺串数互素，最后也能归并到一段，但相邻斐波那契数使各相的分配具有同样的形式，便于安排整个过程。

15.4.2 k 路多相归并

接下来我们将二路多相归并的斐波那契分配推广到 k 路多相归并。对于 k 路多相归并，我们同样希望预先安排好各条磁带的段数，使得能归并到只剩一段。在二路归并中我们使用了斐波那契数列，自然地， k 路时我们可以把斐波那契数列的两项之和推广为 k 项之和。定义 $F^{(k)}(0) = \dots = F^{(k)}(k - 2) = 0$ ， $F^{(k)}(k - 1) = 1$ ，以及

$$F^{(k)}(n) = \sum_{j=1}^k F^{(k)}(n - j), \quad n \geq k.$$

第一步拆分后，每个磁带上的顺串个数应当分别为

$$\begin{aligned} \text{磁带1: } F^{(k)}(n) &= F^{(k)}(n - 1) + F^{(k)}(n - 2) + \dots + F^{(k)}(n - k) \\ \text{磁带2: } &F^{(k)}(n - 1) + F^{(k)}(n - 2) + \dots + F^{(k)}(n - k + 1) \\ &\dots \\ \text{磁带}k: &F^{(k)}(n - 1) \\ \text{磁带}k + 1: &0 \end{aligned}$$

然后合并磁带 1 到 k 的前 $F^{(k)}(n - 1)$ 个顺串，将合并后的结果放到磁带 $k + 1$ 上，此时不同磁

带上的顺串个数变为

$$\text{磁带1: } F^{(k)}(n-2) + F^{(k)}(n-3) + \cdots + F^{(k)}(n-k)$$

$$\text{磁带2: } F^{(k)}(n-2) + F^{(k)}(n-3) + \cdots + F^{(k)}(n-k+1)$$

...

$$\text{磁带}k: 0$$

$$\text{磁带}k+1: F^{(k)}(n-1) = F^{(k)}(n-2) + F^{(k)}(n-3) + \cdots + F^{(k)}(n-k-1)$$

此时的顺串个数分布与之前的是类似的，只是下标减了1，因此可以继续合并除磁带 k 外每个磁带的前 $F^{(k)}(n-2)$ 个顺串，将合并后的结果放到磁带 k 上，以此类推，直到顺串个数为1，此时就完成了排序。

15.4.3 哑顺串与算法实现

实际生成的顺串数量往往无法恰好按上述公式分配。例如，二路归并有30个顺串，而我们希望按 $21 + 13$ 分配，就还差四段。我们可以补充四个长度为0的空段，称为哑顺串。它们只在顺串目录中占一个位置，不需要存放记录。归并时，如果非哑顺串与哑顺串归并，直接复制非哑顺串即可，若两条都是哑顺串，就在输出目录中登记一个新的哑顺串，这样就能保证每次归并的段数与预先安排的分配一致。

15.5 双缓冲

前面通过减少归并趟数来减少外存读写，但每一趟归并本身还有改进的空间。归并需要在内存中比较记录，也需要从外存读入数据、把结果写回外存。如果每次读写都要让归并停下来等待，就会浪费不少时间。下面讨论怎样让这两部分工作同时进行。

先看普通的二路归并如何使用内存。我们为两个输入顺串各准备一个能容纳一块数据的缓存区，从每个顺串中读入一块，再准备一个同样大小的输出缓存区。每次比较两个输入缓存区中尚未处理的第一条记录，把较小的放进输出缓存区，并继续比较。这些比较都在内存中完成，只有某个输入缓存区用完时，才需要读取对应顺串的下一块。输出缓存区装满后，才将其中的记录一起写回外存。这样就避免了每输出一条记录都访问一次外存。

然而，这种方法仍然需要等待。例如，输出缓存区满了以后，必须先将其中的数据写回外存，才能腾出空间接收后续结果。在写完之前，归并就无法继续输出。解决办法是准备两个输出缓存区：第一个满了，就让它开始写回外存，同时继续归并，把新结果放进第二个。等第二个也满了，如果第一个已经写完，就交换两者的用途，继续处理。

输入也可以采用同样的方法。每路只有一个输入缓存区时，只要其中一路的数据用完，而这个顺串还没读完，就必须停下来读取下一块。因此，为每一路准备两个输入缓存区，一个提供当前要比较的数据，另一个提前读入下一块。当前缓存区用完后，如果下一块已经读好，就立即换用另一个，并用刚空出的缓存区继续预读。这种让两个缓存区交替使用的办法称为双缓冲。图15.2给出了这一思想的示意图。

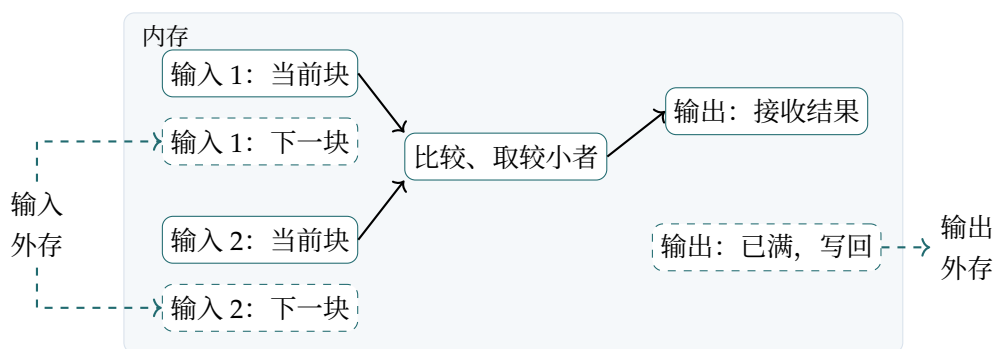


图 15.2 二路归并使用双缓冲时的一个时刻，虚线表示外存读写。

由此可见，二路归并需要四个输入缓存区和两个输出缓存区，一般的 k 路归并则需要 $2k$ 个输入缓存区和两个输出缓存区。

例题 15.2 (双缓冲的内存需求). 有 3250 条待排序的记录，生成顺串时每次可以在内存中排序 750 条。归并时总内存可容纳 750 条记录，每块有 250 条记录。忽略堆等管理开销，能否用双缓冲完成二路归并？

解答. 初始会得到五个顺串，前四段各有 750 条，最后一段有 250 条。归并时，750 条记录的内存只能放下三个块，刚好供两个输入缓存区和一个输出缓存区使用。若改用双缓冲，就需要六个块，即至少能容纳 1500 条记录的内存，因此题设内存不足。□

这也解释了为什么归并路数并非越多越好。总内存不变时， k 越大，需要的缓存区越多，每个缓存区能分到的空间就越小，一次能够成批读写的数据也就越少。即使处理同样多的记录，也可能需要发起更多次读写请求。在机械硬盘上，频繁切换不同输入的位置还会增加累计的寻道时间。因此，增加归并路数虽然能减少趟数，却可能增加每一趟的读写开销，总时间未必缩短。所以，选择归并路数时，既要考虑能减少多少趟，也要考虑可能增大的读写请求，并且给每一路留下足够的缓存空间。

15.6 替换选择

15.6.1 算法描述

还有一个自然的优化方向是扩展初始顺串的长度。如果我们能生成初始更长的顺串，那么初始顺串的数量可能降低，从而可能减少归并趟数。在此前的讨论中，每个顺串的大小都是内存的大小，然而事实上这一长度还可以进一步扩展。原因在于某个内存位置中的记录一旦写入磁盘了，这个内存位置就可以空出来给数组后续的元素使用。只要后续的元素比现在写入的更大，就可以继续往顺串后面添加该元素。这就是替换选择 (replacement selection) 的思路。

为了实现这一思路，我们可以在内存中维护一个最小堆。每次从堆中取出最小的元素写入磁盘后，再从磁盘中读入一个新的元素放入堆中。如果新读入的元素不小于当前顺串中最后一个元素，就可以将其加入堆，继续参与本段的排序。否则这个元素不能加入本段，因此将其暂时冻结。如此继续，直到堆中没有可用的元素为止，当前顺串就结束了。接下来将冻结的元素重新建堆，开始下一段的排序。需要注意的是，堆和冻结部分都占用内存，两者的总大小不能超过内存的容量。

以图 15.1 的例子作为输入进行替换选择，内存大小 $m = 3$ ，开始时先读入 81, 94, 11 建立最小堆，堆顶为 11，此时没有冻结记录。表 15.3 从这一初始状态开始，列出完整过程，后续每行记录一次输

出并补入新元素后的状态，堆中记录按从小到大列出，便于查看。以表中的第一段为例，先输出 11，再读入 96。因为 $96 > 11$ ，96 可以加入堆，继续参与本段的排序。接着输出 81，却读入了更小的 12，只能将 12 冻结，留到下一段。随后读到的 35 和 17 也分别小于刚输出的 94 和 96，因此都要冻结。输出 96 后，堆中已无可记录，于是第一段结束，用冻结的 12, 35, 17 建堆并开始第二段。最终得到 (11, 81, 94, 96), (12, 17, 28, 35, 41, 58, 75, 99), (15)。同样的输入，原来生成五段，现在只生成三段。如果采用三路归并，接下来只需一趟就能完成排序。

表 15.3 对图 15.1 的输入进行替换选择， $m = 3$ 。

顺串	输出	新读入	堆中记录	冻结记录
初始	—	81, 94, 11	11, 81, 94	—
1	11	96	81, 94, 96	—
1	81	12	94, 96	12
1	94	35	96	12, 35
1	96	17	—	12, 17, 35
2	12	99	17, 35, 99	—
2	17	28	28, 35, 99	—
2	28	58	35, 58, 99	—
2	35	41	41, 58, 99	—
2	41	75	58, 75, 99	—
2	58	15	75, 99	15
2	75	输入已读完	99	15
2	99	输入已读完	—	15
3	15	输入已读完	—	—

15.6.2 平均顺串长度

接下来我们讨论替换选择能生成多长的顺串。自然地，这取决于待排序记录的输入顺序。如果输入已经递增，新读入的记录就始终能接在后面，所有记录可以组成一个顺串。如果输入严格递减，新读入的记录就都会被冻结，除最后可能不足 m 条外，每个顺串仍然只有 m 条记录。对于随机输入，有一个常用结论：内存能容纳 m 条记录时，生成的顺串平均约有 $2m$ 条记录。这里假定各条记录的键独立同分布，且服从连续分布，因此不同记录的键相等的概率为 0。下面说明这个结论为什么成立。

为了计算一个顺串平均有多长，我们先把记录分成 m 组，再分别计算每组贡献多少条记录。分组办法是：给最初读入的 m 条记录分别标上 $1, \dots, m$ ，以后每输出一条记录，新读入的记录就沿用输出记录的编号。同一编号的记录归为一组，并按读入先后排列。因此，每组在内存中始终有一条记录，如果新记录比刚输出的同组记录大，就能继续参加当前顺串的排序，否则被冻结，这一组在当前顺串中的输出也随之结束。等所有组都被冻结，当前顺串才结束。因此，**每个顺串从每组中各取出一个连续递增的片段，顺串长度就是这 m 个片段的总长度**。接下来只需证明，每组的递增片段平均有两条记录。注意本节的主要结论和证明都针对输入长度趋于无穷的情况。

注意，新记录属于哪一组，在读取它之前就已经由刚输出记录的编号确定，因此分组不会根据新记录的值作选择。因此，我们可以认为每一组中的记录都是独立同分布的随机数。任取一组，将其中的记录按读入先后记为 $X_1, X_2, \dots$ ，根据独立同分布且分布连续的性质，我们有 $\Pr(X_i > X_{i+1}) = 1/2$ 。

令 I_i 在 $X_i > X_{i+1}$ 时取 1，否则取 0，并记前 K 条记录之间的下降次数为

$$D_K = \sum_{i=1}^{K-1} I_i.$$

相邻的 I_i 会用到同一个随机数, 但 $I_1, I_3, \dots$ 分别使用互不重叠的数对 $(X_1, X_2), (X_3, X_4), \dots$, 所以这一组中的变量独立同分布, 均值为 $1/2$ 。同理, $I_2, I_4, I_6, \dots$ 也满足这个条件。大数定律说明, 独立同分布的 0-1 变量不断取平均时, 平均值以概率 1 趋于其期望。分别用于这两组, 再将下降次数相加, 便得到

$$\frac{D_K}{K} \rightarrow \frac{1}{2} \quad (K \rightarrow \infty), \quad \text{以概率 1 成立。}$$

设前 r 个完整递增片段共有 T_r 条记录, 则第 r 个片段在 X_{T_r} 处结束, 下一项 X_{T_r+1} 比它小, 所以前 $T_r + 1$ 条记录之间恰好发生了 r 次下降, 即 $D_{T_r+1} = r$ 。随着 r 增大, $T_r + 1$ 也趋于无穷, 因而在上述极限成立的序列上, 有

$$\frac{r}{T_r + 1} = \frac{D_{T_r+1}}{T_r + 1} \rightarrow \frac{1}{2}, \quad \frac{T_r}{r} \rightarrow 2.$$

这就证明了同一组中完整递增片段的平均长度趋于 2。前 r 个顺串恰好包含每组的前 r 个片段, 因此把各组的平均贡献相加, 就得到顺串的平均长度趋于 $2m$ 。

上面计算的是许多顺串的平均长度。如果只问第一段的期望长度, 结果可能不同, 因为第一段直接从初始记录开始, 后面的段则从被冻结的较小记录开始。下面用 $m = 1$ 的情况具体算一次。

例题 15.3. 在上述随机输入假设下, 如果 $m = 1$, 生成的第一段平均有多长?

解答. 当 $m = 1$ 时, 第一段会持续输出, 直到新读入的值小于刚输出的值。因此, 第一段长度 L 至少为 j , 恰好要求前 j 个输入严格递增。这 j 个值的 $j!$ 种相对次序等可能, 只有一种递增, 所以

$$\Pr(L \geq j) = \frac{1}{j!}.$$

从而得到

$$\mathbb{E}L = \sum_{j \geq 1} j \Pr(L = j) = \sum_{j \geq 1} \Pr(L \geq j) = \sum_{j \geq 1} \frac{1}{j!} = e - 1.$$

所以第一段的期望长度为 $e - 1$, 小于平均长度 2。□

15.7 最优归并顺序

替换选择产生的顺串长短不一, 合并顺序也会影响总工作量。因为每参加一次归并, 一段中的全部记录就要重新读写一次, 我们希望较长的顺串尽量少参加归并。

如何找到代价最小的合并顺序? 我们可以把初始顺串画成叶子, 每合并两段就给它们添加一个父结点, 从而整个方案便对应一棵每个内部结点都有两个孩子的二叉树。从一个叶子到根, 每经过一个父结点, 它对应的顺串就参加一次归并。因此, 若初始顺串长为 w_i , 对应叶子的深度为 d_i , 这些记录共被处理 d_i 次, 贡献的代价为 $w_i d_i$ 。将各段的贡献相加, 得到总代价 $C = \sum_i w_i d_i$ 。这正是构造哈夫曼编码时要最小化的带权外部路径长度, 所以仍然可以每次合并最短的两段。图 15.3 给出了合并长度分别为 2, 4, 5, 15 的四个顺串的最优二路合并树。

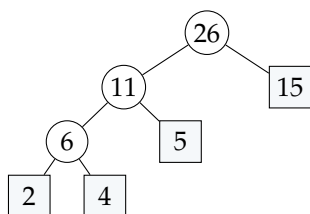


图 15.3 长度 2, 4, 5, 15 的最优二路合并树。

15.8 习题

习题 15.1 (趟数与缓冲). 有 10000 条待排序的记录, 生成顺串时每次可以在内存中排序 1000 条。归并阶段的内存可容纳 1200 条记录, 每个块存放 100 条记录。本题忽略堆等管理开销。

1. 最初生成多少个顺串? 分别采用二路和五路归并, 排序总趟数分别是多少?
2. 若使用单缓冲, 最多能做几路归并? 若全部改用双缓冲, 最多又能做几路?

习题 15.2 (三路多相归并的初始分配). 若有 31 个顺串, 准备用四条磁带做三路多相归并。初始非空的三条磁带的顺串数有以下四种分配方案:

A. (13, 11, 7) B. (11, 10, 10) C. (13, 10, 8) D. (15, 9, 7)

这四种方案中, 哪些不重新分配顺串就能完成排序? 其中哪一种需要的归并相数最少?

习题 15.3 (最优合并). 有 $n \geq 2$ 个顺串, 长度分别为 $1, 2, 4, \dots, 2^{n-1}$ 。每次可以任选两段归并, 代价为两段长度之和。按哈夫曼编码的方法, 应该依次合并哪两段? 共合并多少次, 总代价是多少?

习题 15.4 (替换选择). 1. 假设内存容量为 $m = 3$, 输入 8, 7, 6, 5, 4, 3, 2, 1 会生成哪些顺串?

2. 如果所有输入记录的键都相等, 替换选择会生成几段顺串?
3. 初始顺串数减半, 是否一定能减少归并趟数? 若不一定, 请举出具体例子。

15.9 参考答案

习题 15.1. 初始有 $10000/1000 = 10$ 个顺串。二路归并需要 $1 + \lceil \log_2 10 \rceil = 5$ 趟, 五路归并需要 $1 + \lceil \log_5 10 \rceil = 3$ 趟。单缓冲时, k 路输入加一路输出共需 $100(k+1)$ 条记录的空间, 因此最多做 11 路归并。双缓冲需要 $200(k+1) \leq 1200$, 最多做五路归并。 □

习题 15.2. A、D 都能完成, 归并过程分别需要 5 相和 6 相, 因此 A 最少。 □

习题 15.3. 共合并 $n-1$ 次, 第 j 次的代价为 $2^{j+1}-1$, 总代价为 $\sum_{j=1}^{n-1} (2^{j+1}-1) = 2^{n+1} - n - 3$ 。 □

习题 15.4. 递减输入生成 (6, 7, 8)、(3, 4, 5)、(1, 2)。全相等输入只生成一段, 因为新记录总是不小于刚输出的记录, 可以继续留在当前段。五路归并时, 320 段和 160 段都需四趟, 说明段数减半不一定减少归并趟数。 □

15.10 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第十五讲。
2. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*, 第二版, 第 7.11 节介绍外部排序, 包括多路归并、多相归并和替换选择。
3. W. C. Lynch, *The t -Fibonacci Numbers and Polyphase Sorting*, *The Fibonacci Quarterly*, 8(1):6–22, 1970。原论文第 5–6 节介绍归并与多相调度, 第 7 节进一步处理未知初始段数的分配问题。
4. Kurt Mehlhorn、Stefan Näher, *LEDA: A Platform for Combinatorial and Geometric Computing*, 1999, 第 3.5 节用随机输入实验讨论替换选择的顺串长度, 并介绍了相应的扫雪机模型。
5. Jeffrey Scott Vitter, *External Memory Algorithms and Data Structures: Dealing with Massive Data*, *ACM Computing Surveys*, 33(2):209–271, 2001。作者公开论文第 5.2 节讨论外部归并, 第 5.7 节讨论外部排序所需的 I/O 次数下界。

祝贺大家顺利完成了这门课程的学习, 希望这本讲义能够提供理解上的帮助, 给予一些启发。尽管这门课的学习充满了痛苦, 但我相信只要是平常认真学习了这些内容的就一定能够有所收获。祝愿大家在期末考试中取得好成绩, 再见!